CS 2340 - Ben Holland Term Project

Program Description

This MIPS program is a single player game which follows the same rules and mechanics of the Number Chain game shared in class when the assignment was opened. Each turn, the player inputs a starting cell through direct X and Y coordinates in addition to what direction they want to move in. After the player makes a move, the computer will make a move branching off of a cell it owns, starting its search from a random cell. This cycle continues until the board is completely filled with cells after which the user with the highest value cell is declared the winner, with a congratulation if the player won and a commiseration if they lost.

The program is split into modules that each handle one part of the game: main.asm is the entry point, board.asm holds the board data and handles setting up and drawing the board, gamelogic.asm runs the turn loop and contains the move rules, the computer AI and the final scoring, input.asm reads and validates everything the player enters, and helpers.asm holds the shared constants and macros.

Challenges Faced

The first major challenge I faced was how I was going to represent the links between cells on a strictly ASCII board. Due to the fact that a link can go horizontally, vertically, or diagonally, I actually had to scrap my first iteration of the game that strictly printed the values in a grid with no borders surrounding each cell. How I ended up going about fixing the link problem was having three separate link arrays in addition to the regular values array. H_links marks a link between a cell and its neighbor to its right, which gets drawn using a - instead of the usual |. V_links marks a link between a cell and the one above it, which swaps the - - - border with |. Finally, d_links handles diagonals at the junctions between rows, where 1 means / and 2 means \. The vertical and diagonal arrays are 7x6 instead of 6x6, since there's always going to be one more border line than there are rows. Using these arrays, I added the set_link procedure which takes any two neighboring cells in any order, sorts them so that the upper or left one comes first, and then works out which array and slot to write to. This system does have one limitation, which is that a / and a \ crossing through the same junction can't both be rendered simultaneously, since that junction only gets one character to display the linkage.

The second challenge was the math needed for converting between 2D coordinates and a flat 1D array. MIPS doesn't give you multidimensional arrays, so every board array is just a buffer of 36 words. In order to get a cell, the byte offset works out to (row * 6 + col) * 4, which gets added

to the calc_offset, get_cell, and set_cell macros so that the rest of the code could use regular row and column coordinates instead of having to constantly calculate out the offsets. Conversely, the enemy AI needed to go the other direction, with it finding 2D coordinates from a raw index value. The math works out to row = (i / 6) and col = (i mod 6), with the row being pulled out of the lo register and the column out of hi.

The third challenge was tracking down edge cases in input handling and winner validation. For input, SysReadString holds the newline from hitting Enter, so the direction match strings need to be stored as “d\n”, “ur\n”, and so on, or nothing would ever be properly matched to what the user inputs. The coordinates also come in as X then Y while the board is indexed as row-major, so the X has to be treated as the column and Y as the row, which is insanely easy to flip on accident while working in assembly where there are no variable names. With the game flow side, I had to figure out how to deal with one side running out of moves before the board fills up. If the player is stuck but the computer isn’t, the computer just keeps going until a winner is declared, and if the computer is stuck, the turn goes back to the player. The game only ends once nobody can move.

Lessons Learned

The biggest thing I learned is just how much I lean on structured data definitions in higher-level languages without even thinking about it. In an object-oriented programming language, each cell would just be an object with a value, an owner, and its links, and all I would need to do is have a 2D array of those objects which get printed out. MIPS (or any assembly language that I know of) doesn’t gives us any out of the box way of defining structured data, so I had to change how I approached the whole problem and split every “cell” across a handful of parallel arrays (those being grid, owners, h_links, v_links, and d_links) that all share the same indexing scheme. Keeping all of those in sync by hand is certainly doable, but gets very confusing and painful once a program grows past a certain size.

I also learned how much macros and constants can do when you don’t have real abstractions to work with. Defining board size and the array sizes with .eqv and hiding the offset calculations inside macros meant most of the game logic sometimes reads pretty close to pseudocode, which made it a lot easier to come back to after not touching it for over a week at a time. Additionally, I learned to be a lot more careful with registers, since macros like get_cell and set_cell internally modify registers like $t8, I had to keep a running list of what registers are doing at any given time to avoid accidentally overwriting something I still needed. Overall, I tried to structure my program like C with a main function that essentially acts as an entry point, which then calls the relevant functions in a structured manner.

Algorithms & Techniques

I've split this section of the report into the four main sections that this project called for.

Input Validation

A player's turn starts by reading the X and Y coordinates from 0 to 5 using read_coord, which reads each one as text with SysReadString and converts a single digit from ASCII, returning -1 for anything else so invalid input is rejected instead of crashing. From there, verify_starting_position checks that the coordinates are actually on the board, that the cell isn't empty, and that the player owns it. If any of these steps fail, the program prints an invalid start message and sends the player back to the coordinate prompt to try again.

Next, the direction gets read into a 12-byte buffer with SysReadString and converted to lowercase by lowercase_dir, so "UR" works the same as "ur". That input is then checked against every valid direction code using the compare_string macro, which walks both strings byte by byte until it hits a mismatch or reaches the end of both. Once something matches, the program jumps to the specified handler procedure which loads the row and column offsets for that direction, and if nothing matches, it prints an invalid move message. In a higher level language, I think I could simplify this down by assigning an X and Y offset value to each character type (L = X - 1, U = Y - 1, etc), and then just parse out the string and match to their offsets. But due to the lack of high level standard libraries to pull from, the cleanest approach I could think of was essentially a convoluted switch statement.

Finally, the move itself gets checked in try_move. It works out the target cell from the origin plus the offsets, and it rejects the move if the target is off the board or if the target is already set to a value that is nonzero, which keeps anyone from overwriting a cell. If the move is legal, the target gets set to the origin's value + 1, the target's owner is copied from the origin's owner, and set_link connects the two cells. A rejected move prints a message, redraws the board as it was, and then asks the player for a new input.

Board Rendering

All of the drawing happens in print_board, which prints the board as alternating border lines and cell lines, so a 6x6 board ends up being 7 border lines with 6 cell lines in between, plus a header of column numbers across the top. Each border line starts with " +" and walks across the six columns, reading v_links to pick between - - - for a normal wall and | for a vertical link, and then reading d_links to pick the junction character to the right of that column, where 0 gives +, 1 gives / and 2 gives \. Since a diagonal link always crosses through where four cells meet, putting it in the border line means the linkage character is right where your eyes would be expecting it. After the seventh border line prints, the row counter equals the board size, so the procedure exits instead of trying to print a seventh row of cells, which is how the bottom border shows up without needing its own special case.

Each cell line starts with the row number, which gets printed by adding 48 to the row, since 48 is the ASCII code for “0” and the digits come right after it. Then, for every column, it reads the cell’s value from grid and its owner from owners, where the owner value picks ‘P’ or ‘E’ out of the owner_chars table, and the value from grid gets printed after it, with a space in front if it’s a single digit. After the label prints, it reads h_links to choose between the normal | wall and a - for a horizontal link to the cell on the right. An empty cell just prints three spaces, and the padding on single digit numbers is why every cell is exactly three characters wide, which keeps the whole grid lined up no matter what’s in it. The border segments use a fixed-size trick instead, where “---” and “ | “ are each exactly 4 bytes long (three characters and the null terminator), so finding either one is just address = base + value * 4.

In a higher-level language, I could just convert the number to a string and concatenate the letter onto the front of it, but I wasn’t able to implement that in MIPS. My solution was to print the letter, the padding, and the number one after another so they show up as a single label. Print_board also uses the $t registers and never calls another procedure, so it doesn’t need to save anything on the stack and won’t mess with any $s registers its caller is still using. It gets called at the start of the game, after every player and computer move, and after a rejected move so the player can look at the board before trying something else, with a dividing line of equal sign characters printed before each new board to keep things readable while scrolling back through the console output.

Enemy AI

The computer’s turn starts by picking a random cell index from 0 to 35 (LAST_CELL) and then scanning every cell on the board starting from there. When it gets past cell 35, it wraps back to 0 by taking the remainder by CELL_COUNT, and a counter that starts at CELL_COUNT counts down so every cell only gets accessed once. This way, if there's a legal move anywhere on the board, the computer will always end up finding it. Since the scan goes through a flat 1D index instead of rows and columns, I had to pull the row and column back out by dividing by 6, which is the same thing I talked about earlier in the challenges section, and any cell that’s either empty or owned by the player just gets skipped over.

For every cell the computer owns, it tries all 8 directions in a set order using two word arrays, dir_row_ofs and dir_col_ofs, where each index in both arrays gives the row and column offsets for one direction. Each attempt gets passed into the same try_move procedure the player uses, so I didn’t have to write a separate set of rules for the computer, and it still has to pass the same bounds check and can’t overwrite a cell either. Once a direction works, try_move has already given the new cell to the computer, so computer_move prints the board and returns. A stuck computer never reaches this point, because game_loop checks owner_has_move before calling it. The randomness comes entirely from where the scan starts and not from the direction, which keeps the computer from building off the same chain every turn, but it also means the direction it picks always follows the same priority of up, down, left, right, and then the diagonals.

Winner/Loser Validation

Before every turn, game_loop calls owner_has_move for the side whose turn it is to check. The procedure goes through every cell owned by the side whose turn it is and checks all 8 of its neighbors to see if any of them are on the board and empty, without changing anything on the board itself. As soon as it finds an open neighbor, it returns 1, since a side only needs one legal move for their turn to go ahead. It also uses the same direction arrays as the enemy AI, so both procedures are working off the same definition of what a neighbor is.

If that side can't move, the same check gets run for the other side. If the other side can still move, the game prints a message saying who's stuck and the other side takes another turn, and this keeps repeating until one of them runs out of moves too. If neither side can move, the game is over, and at that point the board has to be completely full, since an empty cell next to any filled cell would be a legal move for whoever owns the filled one. Because of this I didn't need a separate check for whether the board is full, since checking if either side can move already handles it for me, along with the case where only one side gets boxed in early.

Once the game ends, game_over goes through the whole board once and keeps track of two maximums: one for the player's highest value and one for the computer's, using the owners array to decide which one each cell gets compared against. It prints both numbers, then congratulates the player if they're higher, commiserates if the computer is higher, or announces a tie, and returns to main, which exits the program.